



Transactions — Chapter 17

2025年12月

School of Computer Science



北京邮电大学

Part 5 TRANSACTION MANAGEMENT

- **单用户**对DB访问体现为DBS的**1个或多个事务(transaction)**执行
 - select, insert, delete, update
 - **事务**是DBAS**基本逻辑单位**，是DBMS管理DBS的基本单位
- **多用户**对DB**并发访问**体现为DBS的**1个或多个事务**并发执行

TRANSACTION MANAGEMENT

- 多事务对共享数据Q并发访问，为数据正确性(integrities)，满足ACID特征：
原子性(atomicity)，
一致性(consistency)，
独立性/隔离性(isolation)，
操作结果 持续性/永久性保持 (durability)
- DBMS的recovery-management 负责原子性和永久性
- DBMS的concurrency-control 负责独立性和一致性

17.1 Transaction Concept

- **Def 1:** The transaction is a unit of **DBAS**
 - executing **accesses** and **updates** various data items in DB
- **Def 2:** A transaction consists of **operations** delimited by statements
 - **begin** transaction and **end** transaction
 - **operations:** **read** or **write** on DB
 - **begin** transaction
 - $op_1; op_2; \dots$
 - **end** transaction (**commit** or **abort/rollback**)
- A transaction is **initiated** by an **application program** written in
 - DML, such as SQL
 - programming languages, such as C++ embedded accesses

Transaction Concept

- **Transaction:** defined by **read** and **write meta** operations 【逻辑读、逻辑写】

- **e.g.1. Account-transfer** T1 : transfer \$50 from account A to account B

T1 : **begin-transaction**

read (A);

A := A -50;

write (A);

read (B);

B := B + 50;

write (B);

end-transaction

- **Issues:** deal with

- **failures**, such as hardware failures and system crashes
- **concurrent** execution of multiple transactions

- Transaction with roll back

T2 begin-transaction

read (A);

 A:= A -50;

if A < 0 then

begin

 print('insufficient funds')

rollback T2

end

else begin

write (A);

read (B);

 B := B + 50;

write (B);

commit T2

end

- **rollback/abort T2**: 撤销之前对loan_A 的修改, 即恢复loan_A 的原值, 并立即结束T2, 不再执行其后的各操作

■ SQL statement vs read/write meta operation

- SQL statement can be decomposed into **read** and **write** operations
- e.g. SC(s#, c#, grade)

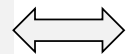
```
insert into MaxGrade(c#, maxG)
  select c#, max(grade)
  from SC
  group by c#
```

```
update SC
  set    grade=grade +5
  where c#=301141
```



```
read(SC(c#, grade))
```

```
write(MaxGrade(c#, maxG))
```



```
read(SC.c#)
```

```
read(SC.grade)
```

```
write(grade)
```



```
DECLARE @transfer_name varchar(10)
SET @transfer_name = 'I-transfer-from-A-to-B'
BEGIN TRANSACTION @transfer_name
USE ACCOUNT
GO
UPDATE A
    SET balance = balance - 50
    WHERE branch_name= 'Brooklyn'
UPDATE B
    SET balance = balance + 50
    WHERE branch_name= 'Brooklyn'
GO
COMMIT TRANSACTION @transfer_name
GO
```

事务变量定义

事务命名

事务开始

打开数据库ACCOUNT

将上述批语句提交SQL Server

修改A帐户/

* 修改B帐户*/

事务提交/

17.2 Transaction Model

- Each data access, such as **select**, **update** in SQL, is translated/decomposed into **read** and **write** operations
- **DBMS:** executes **read** and **write** operations to fulfill **data access** in transactions
- **DBMS:** allocates **local buffer** in **main memory** as working area
- The DB **permanently** resides on disk, but some portion is **temporarily** residing in **buffer** in main memory



Transaction Model

- **Def: read(X)**

- **transfer** data item **X** from DB files on **disk buffer** or **disk** to a **variable**, also called **X**, in the **local buffer**, belonging to a **transaction** that executes the **read** operation

- **Def: write(X)**

- **transfer** the value in **variable X** in **local buffer** back to **data item X** in DB files on **disk**, belonging to a **transaction** that executes the **write** operation

- **Note:**

The **write** operation **does not** result in the **immediate update** of data on **disk**. It may be **temporarily** stored in **disk buffer**.

Transaction

e.g. *name* in *Instructor*

data accesses on data item **x** and **y** issued by T_i ,
e.g. **select**, **insert**, **delete**, **update**, ...

DBMS

variable

x_i, y_i

local buffer for transaction T_i

逻辑读写

read(x)

write(y)

data item

B_x

B_y

...

B_z

disk buffer

物理读写

input(X)

output(Y) / **reflect** / 持久化

when checkpoint or commit

DB file on disk

B_x

B_y

...

B_z

B_w

B_u

B_v

...

B_q

B_r

Fig. **read** and **write** operations

Example of Fund Transfer

- **Example:** Transaction to transfer \$50 from account A to account B:

1. **read(A)**

2. $A := A - 50$

3. **write(A)**

4. **read(B)**

5. $B := B + 50$

6. **write(B)**

A=1000

B=2000

A + B = 3000

A=950

B=2000

A+B=2950

A=950

B=2050

A+B=3000

1. Consistency

- **Execution** of a transaction in **isolation** preserves **consistency** of DB.
DB is in a **correct** DB state, **before** and **after** the transaction execution.
 - 事务的正确执行使得DB从一个正确状态转移到另一个正确状态
 - DB state: contents of tables in DB or DB instance

- **Def: Consistency/correctness**

- **Integrity** constraints are not violated,

- **Example: Consistency requirement**

- The sum of A and B is unchanged
 - i.e. $A + B = 3000$

A=1000,B=2000
A+B=3000

1. **read** (A);
2. A := A -50;
3. **write** (A);
4. **read** (B)
5. B := 50;
6. **write** (B);

A=950, B=2050
A+B=3000

Consistency (Cont.)

- **Consistency requirements:** include
 - ▣ **Explicit integrity** constraints such as primary key and foreign key
 - ▣ **Implicit integrity** constraints
 - sum of balances of all accounts minus sum of loan amounts must equal value of cash-in-hand
 - ▣ The DB may be **temporarily inconsistent** during transaction execution
事务执行过程中，有可能处于不一致状态
 - e.g. $A+B=2950$ after step 3 and before step 6
 - ▣ The DB must be **consistent** when transaction completes **successfully**

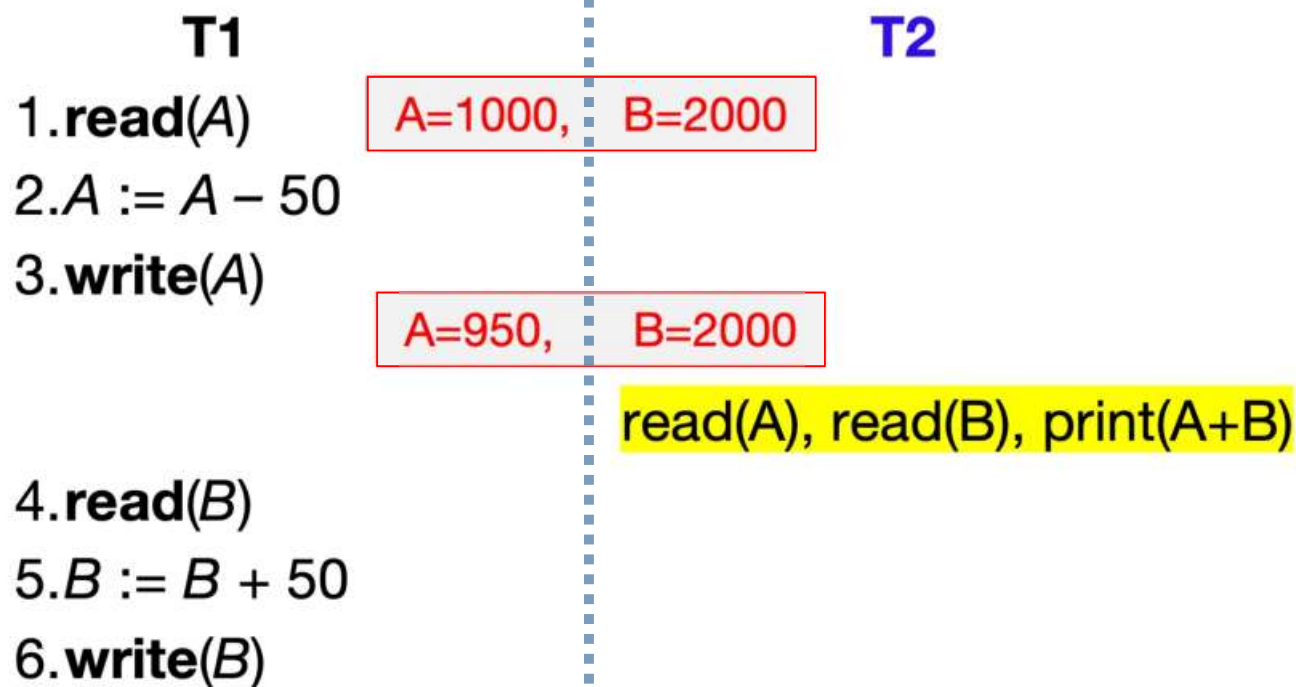
2. Atomicity

- **Either:** all operations of transaction are reflected/persisted 持久化 (▶)
or: none
 - 事务中所有操作或者全部成功完成，并且这些操作结果 被写入到DB;
 - 事务中的所有操作一个都不作，该事务对数据库和其他事务没有任何影响
- Ensure that updates of a **partially** executed transaction are not reflected

3. Isolation

- **Def:** Even though **multiple** transactions execute **concurrently**, all transactions seem to execute **serially**, so the consistence is preserved.
 - each transaction is **unaware** of **other transactions** executing concurrently
 - 对并发执行事务，1个事务执行不能被其他事务干扰，即1个事务内部操作和数据对其它事务是隔离，并发执行事务间不能相互干扰，从执行结果来看，**相当于事务串行执行**
- **Isolation:** guarantees
 - **correctness** of **concurrent execution** of multiple transactions
 - **concurrency control** ensures the isolation

- e.g. if between steps 3 and 6 , another transaction T2 is allowed to access the partially updated DB, it will see an inconsistent database



The Reader-Writer problem

- Isolation can be ensured by running transactions **serially**, one after the other

4. Durability

- **Def:** Once the transaction has completed,
the updates to the database by the transaction must **persist**
even if there are software or hardware failures
 - ▣ The **modified** values of data items in **buffer**
should be reflected/**write** to DB file **on disk**
 - ▣ 事务一旦提交（成功完成）对DB数据改变是永久性—持久化
- ▣ e.g.: the transfer of \$50 has taken place,
the updates to the database by the transaction must persist
even if there are software or hardware failures



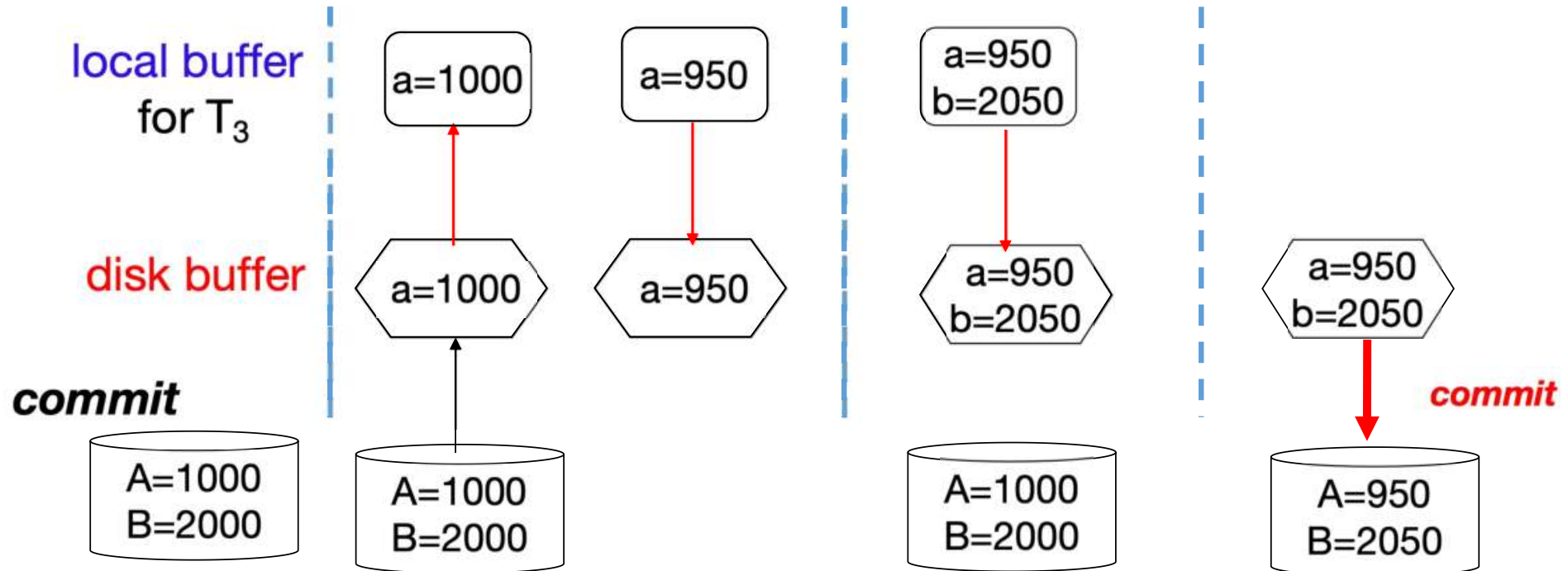
- e.g.2. for T_3 in Fig.



begin-trans.

update(A) ;

update(B) ;



ACID Guaranteeing

- ACID mechanisms in DBS
 - ▣ **Atomicity.** transaction management / recovery management component
 - ▣ **Consistency.** integrity constraints; testing mechanism in DBMS
 - ▣ **Isolation.** concurrency control component
 - ▣ **Durability.** recovery management component

17.4 Transaction Atomicity and Durability (State Model)

- **Def 1:** A transaction is termed **aborted** (夭折),
if it may not complete its execution successfully
- **Def 2:** A transaction is said to be **committed** (提交),
if it completes its execution successfully
- **Def 3:** Once the changes by an **aborted** transaction has been **undone** (撤销),
the transaction is said to be **rollback** (回滚)
- **Def 4:** A transaction is said to have **terminated** (结束),
if it has either committed or aborted

17.4 Transaction Atomicity and Durability (State Model)

- **Transaction state** – during its life-cycle
- **Active** – the initial state
- **Partially committed** – after the final statement commit, executed
- **Failed** – the normal execution can no longer proceed
- **Aborted** – after the transaction has been rolled back
the DB restored to its state prior to the start of the transaction.
Two options after aborted:
 - **Restart** the transaction: only if no internal logical error
 - **Kill** the transaction
- **Committed** – after successful completion

Transaction States

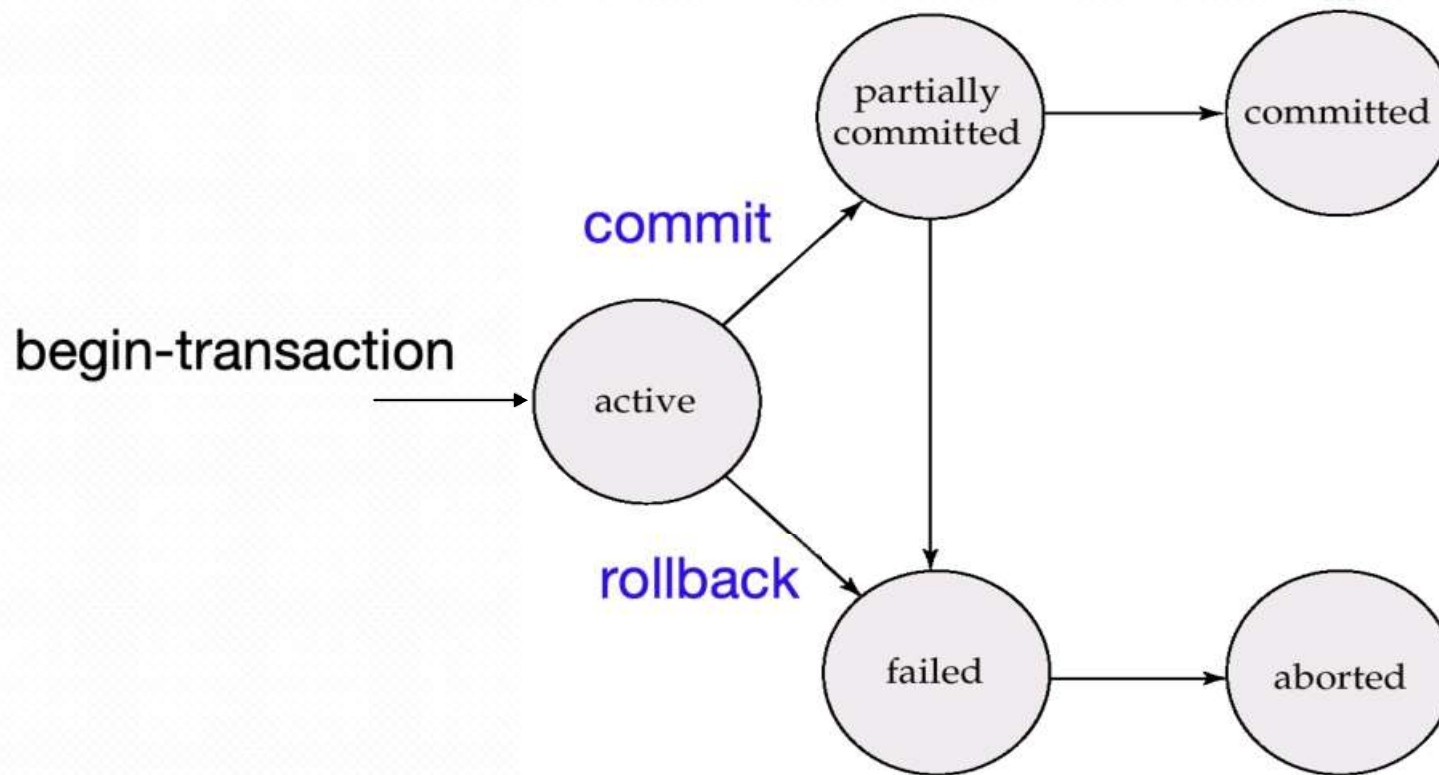
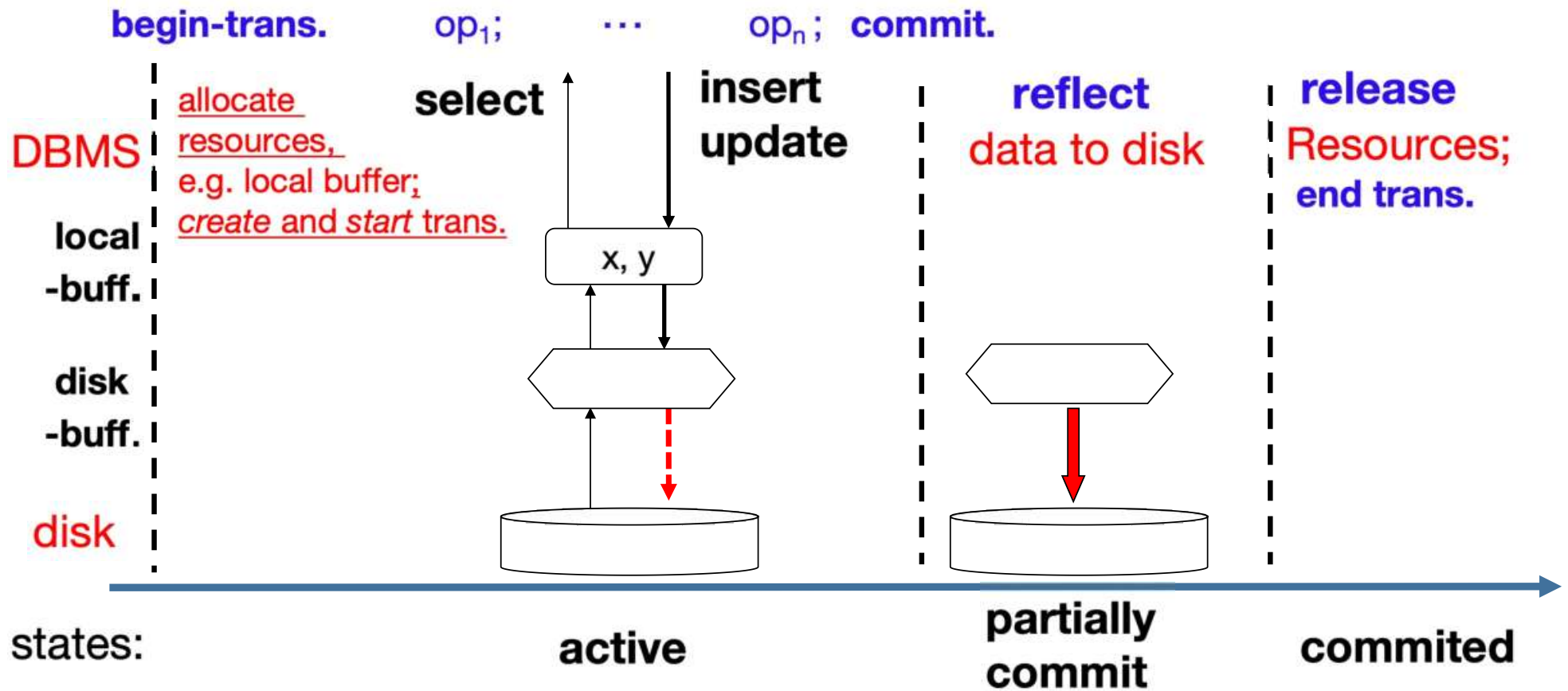


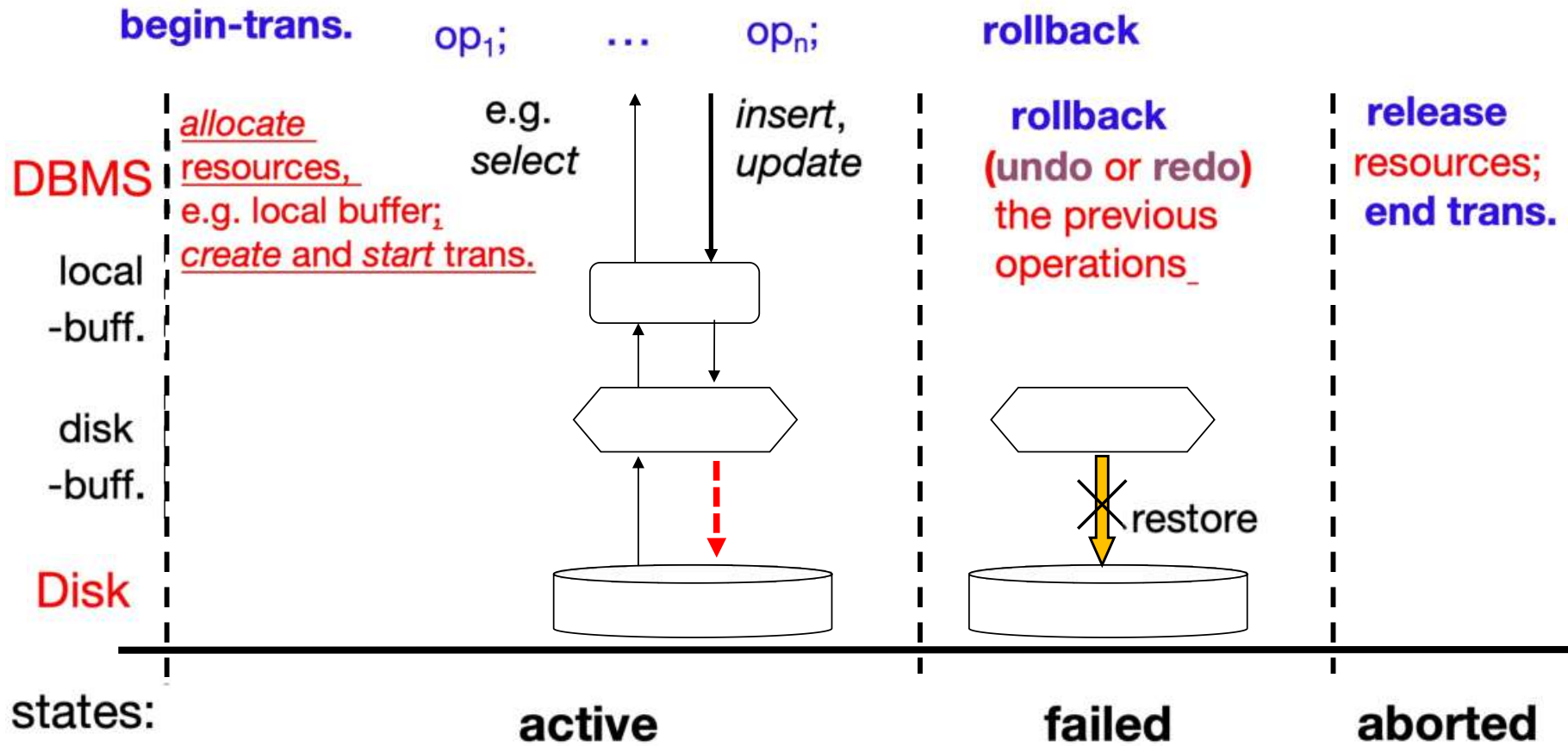
Fig.17.1. State diagram of a transaction

Transaction



A successful/committed transaction

Transaction



A rollback transaction

Transaction States

■ Def: **Active**

- the initial state, the transaction stays while it is executing from *begin-transaction*
- the transaction is created (after **begin-transaction** is submitted)
e.g. **local buffer** are allocated to the transaction

■ Note

- transaction operations are executed, but the modifications on DB issued by operations **may** only stored **temporally** in the disk buffer
- **not reflected** to DB file on disks immediately

Transaction States

- **Def: Partially committed** (部分提交) :

- After the final statement (*Commit*) has been submitted, the transaction enters this state
- The influences that the transaction operations have made are **reflected** from the disk buffer to the DB file

DBMS将disk缓冲区中的DB修改结果写入磁盘DB文件—持久化

Transaction States

■ Def: **Committed**

- after all the operations are successfully completed, it enters this state.
all results have been reflected to the DB file in the **partial commit** state.
- the transaction
 - releases the resources occupied
 - terminated (quits DBS),
 - the cycle-life of the transaction is ended

Transaction States

■ Def: **Failed**

- ❑ If it is discovered that normal execution can no longer proceed, the transaction enters this state
- ❑ The transaction is **rolled back** to restore DB to the state prior to the start of the transactions
- ❑ 在此状态下进行失败处理工作

Transaction States

- **Def: Aborted** (夭折/异常结束/中止状态)
 - **After** the transaction **has been rolled back** in the **failed** state, the DB has been restored to its state **prior to** the start of the transactions, it enters this state to **end** the transaction
 - the transaction
 - **releases** the resources occupied
 - **reports** to its user that the transaction is **aborted**
 - **terminated** (quits DBS)
 - the cycle-life of the transaction is **ended**
 - 表示DBMS已完成失败处理工作, 事务将要退出系统

Example: **Roll Back** in Transaction

- **Def: Roll back** (回滚、滚回)

- ▣ 撤销到目前为止事务已经对数据库做的所有修改,
数据库状态恢复到**事务开始前**的状态

- ▣ e.g. Define table **SC** as

```
create table SC  
  s# integer,  
  c# integer,  
  grade integer,  
  check (grade between 0 AND 100 )
```

Define the *update* transaction
update-tran as:
update SC
set *grade* = *grade* + 5
where

update on some tuples

rollback the tuples updated

s#	c#	grade
101	1	85
102	2	55
201	2	75
210	5	100
401	3	77
...	...	...
510	2	60

(a) the initial
SC instance

s#	c#	grade
101	1	90
102	2	60
201	2	80
210	5	105
401	3	77
...	...	...
510	2	60

(b) **SC** instance when
failure occurs, i.e.
integrity is violated

s#	c#	grade
101	1	85
102	2	55
201	2	75
210	5	100
401	3	77
...	...	...
510	2	60

(c) **SC** instance after **roll back**,
i.e. canceling all updates,
equal to the *initial instance*

17.5 Transaction Isolation

Scheduling on concurrent transactions 调度

- **Concurrent execution** of transactions allows
 - high throughput and resource utilization, **reduced waiting time**
 - e.g. the multi-user ticket-ordering system
- **Demerits** for allowing transaction concurrency
 - transactions contain operations on ***shared data***,
the different schedules may result in **different final DB states**
- **Concurrency-control schemes and transaction schedules are needed**

Concurrent Executions & Scheduling 调度

- **Def: Schedule** on a set of concurrent transactions,
 - specifies the order of the executed operations of concurrent transactions
 - consist of **all operations** of transactions
 - preserve **the order of operations** in each individual transaction
- **Note:**
 - A transaction that **succeeds** in execution will have a **commit instruction** as the last statement
 - A transaction that **fails** in execution will have an **abort instruction** as the last statement

An Example

- T_1 : transfer \$50 funds from A to B
- T_2 : transfer 10% of the balance from A to B
- initial value of A: \$ 1000
initial value of B: \$ 2000
initial (A + B) : \$ 3000

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	 read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

Serial **schedule S1** i.e. $T_1; T_2$

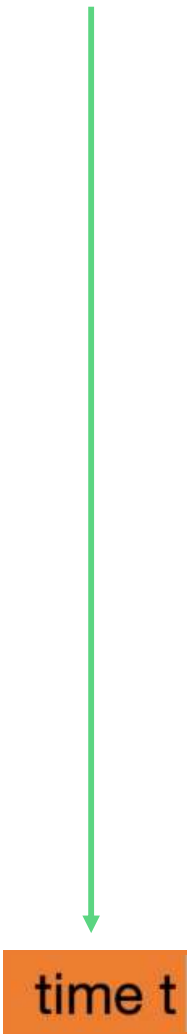
A=950, B=2050

Final results:

A=\$855, B=\$2145

(A + B) = \$3000

T1:	T2	account-A file	account-B file
read (A1)		\$1000	\$2000
A1:= A1-50		\$1000	
write (A1)		\$950	
read (B1)			\$2000
B1:= B1+50			\$2000
write (B1)			\$2050
	read (A2)	\$950	
	temp:= A2*0.1	\$95	
	A2 := A2- temp	\$950	
	write (A2)	\$950	
	read (B2)		\$2050
	B2:= B2 + temp		\$2050
	write (B2)		\$2050
temp=95, Final (A + B) = \$855 + \$2145 = \$3000			

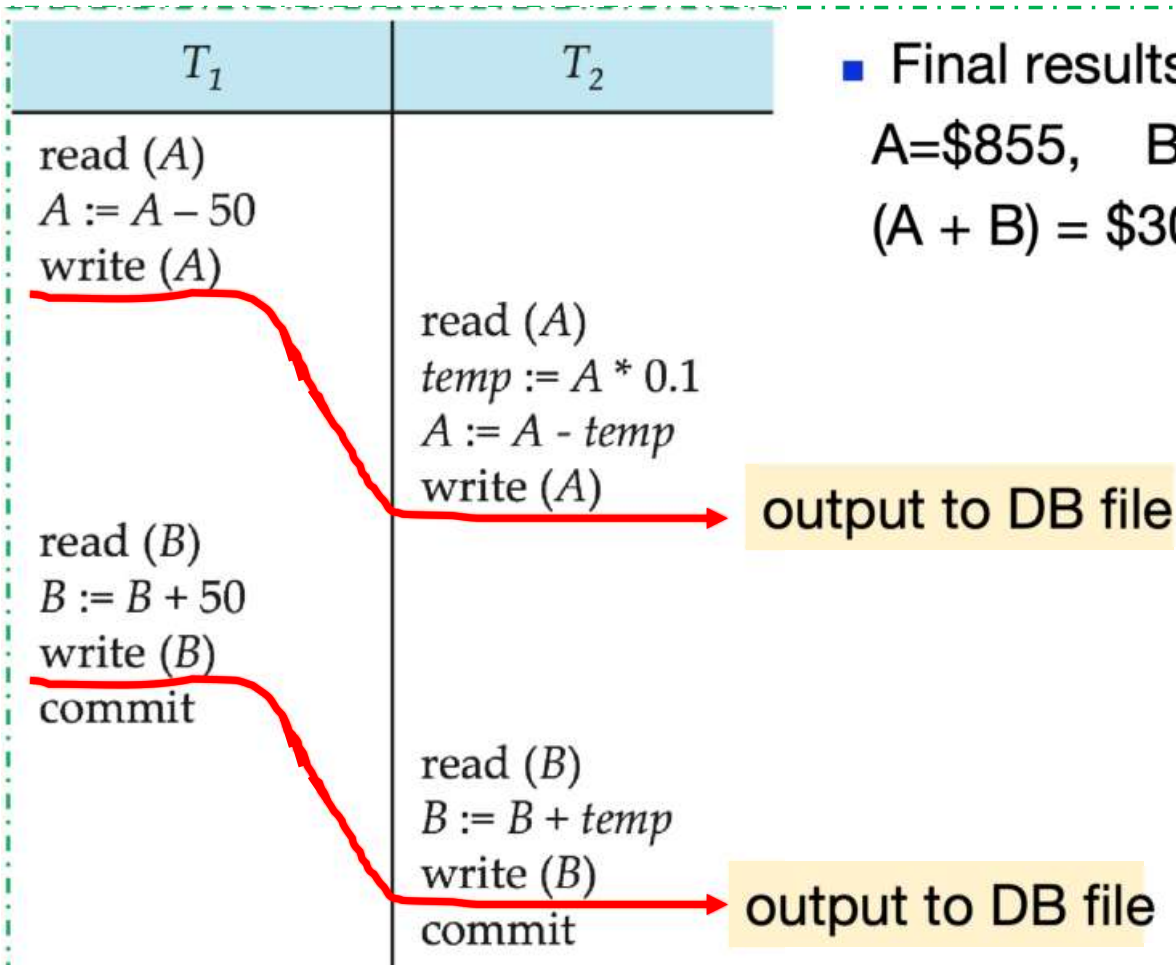


An Example (cont.)

- After T1 and T2 are finished under the constraints of the **schedule S2**,
 - ▣ the sum of A and B is preserved as 3000
 - ▣ the final value of **account-A** is **850**
 - ▣ the final value of **account-B** is **2150**
- Schedule S1 and S2 result in
 - ▣ **different final values** of shared data items A and B,
 - ▣ all **correct** schedules

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

Serial **schedule S2**, i.e. T2; T1



■ Final results

$A = \$855$, $B = \$2145$

$(A + B) = \$3000$

并发调度S3将对A的2次写操作、对B的2次写操作安排在相近时间执行，利于在commit时刻通过一次I/O操作一次性地将A/B修改结果写回数据库文件，减少DB file写/output操作次数

将对A的2次读操作、B的2次读操作安排在相近时间执行，有利于减少对DB file 读/input操作次数：

第2次从disk buffer中读

Fig.17.4 schedule S3

S3:

T1:

T2

account-A DB file

account-B DB file

read(A1)
A1:= A1-50
write(A1)

read (A2) =950
temp:= A2*0.1
A2 := A2- temp
write(A2)

read (B1)
B1:= B1+50
write(B1)

read (B2)
B2:= B2 + temp
write(B2)

\$1000

\$1000

\$1000

\$950

\$950

\$95

\$950

\$855

\$2000

\$2000

\$2000

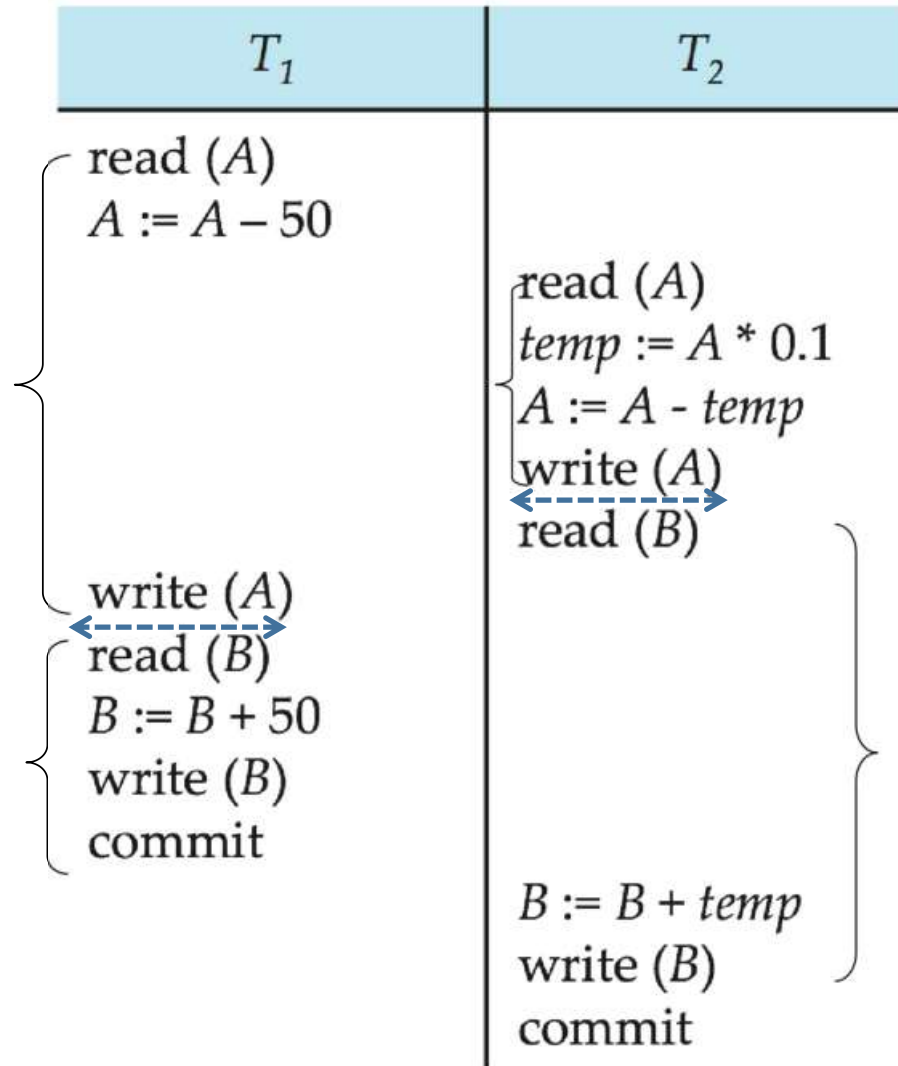
\$2050

\$2050

\$2145

temp=95 , final (A + B) = \$855 + \$2145 = **\$3000**

S4:



■ Note:

assuming, **read** fetches data directly from DB file on disk

■ The final shared results:

$A = \$950$, $B = \$2100$

$(A + B) = \$3050$

Fig.17.5 **Incorrect** concurrent **S4**

S4: T1:

T2

account-A *DB file*

account-B *DB file*

\$1000

\$2000

\$1000

\$1000

\$1000

read(A1)

A1:= A1-50 =**950**

read(A2)=1000

temp:= A2*0.1=**100**

A2 := A2- temp=900

write (A2)

read(B2)=2000

\$900

\$950

write (A1)

read(B1)

B1:= B1+50 =**2050**

write (B1)

\$2000

\$2000

\$2000

\$2050

B2:= B2 + temp

write(B2)

\$2100

A1=950, temp=100, final (A + B) = \$950+ \$2100 = **\$3050**

Why S3 gives a correct result, while S4 leaves a wrong state ?

■ S3 (**equivalent**) S1

- T1 and T2 access the shared data **A** and **B** **serially**
- S3 has the same executing results as serial S1
- S3 is **equivalent to** serial S1

■ S4 (**not equivalent**) S1

- T1 and T2 operate on shared data **A** and **B** in **interweaving** ways
- S4 has not the same executing results as serial S1,
- S4 is **not equivalent to serial S1**
- S4 leaves the DB in an **inconsistent** state,
- **Note: isolation** and **consistency** are violated

17.6 Serializability

- 1. 多事务**串行执行**可保证事务ACID特性 (S1, S2) , 但**执行效率低**
- 2. 多事务**并发执行**时, 如果事务操作调度顺序不当 (S4) , 影响DBS正确性
- 事务并发控制基本原理: 事务调度可串行化
 - 相互**冲突操作**串行执行
 - 非**冲突操作**并行/交织执行
- **Motivation: Serializability 可串行化**
 - 可串行化: 判断**并发调度**是否正确的标准 (S3)

Serializability

- **Identify:** whether a **concurrent schedule S** on transactions is **right**
 - S is **equivalent** to a **serial S'**
- **Basic Assumption:**
 - **each transaction** preserves DB **consistency**
 - **serial execution** of **transactions** preserves DB **consistency**
- **Def:** A schedule S is serializable if it is **equivalent** to a **serial schedule S'**.
- **Serializable schedule (**Serializability**)**
 - conflict equivalence/**conflict serializability**
 - **view** equivalence/**view serializability**
 - 将对S正确性判断**转换**为对S可串行性的判断

Conflict Serializability

- Two transactions $T_i = \{ I_i \}$, $T_j = \{ I_j \}$, and schedule S on T_i and T_j
- **Def:** I_i of transactions T_i and I_j of T_j are **conflict**, if and only if
 - for some item Q accessed by both $I_i(Q)$ and $I_j(Q)$, at least one of these two instructions is **write**(Q)
 - **i.e.**
 - $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$
 - $I_i = \text{write}(Q)$, $I_j = \text{read}(Q)$
 - $I_i = \text{write}(Q)$, $I_j = \text{write}(Q)$

$I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$,
 I_i and I_j don't conflict.

Conflict Serializability

- **Def:** S and S' are **conflict equivalent**
 - if the schedule S can be transformed into S' by a series of swaps of **non-conflicting** instructions/operations
 - e.g.

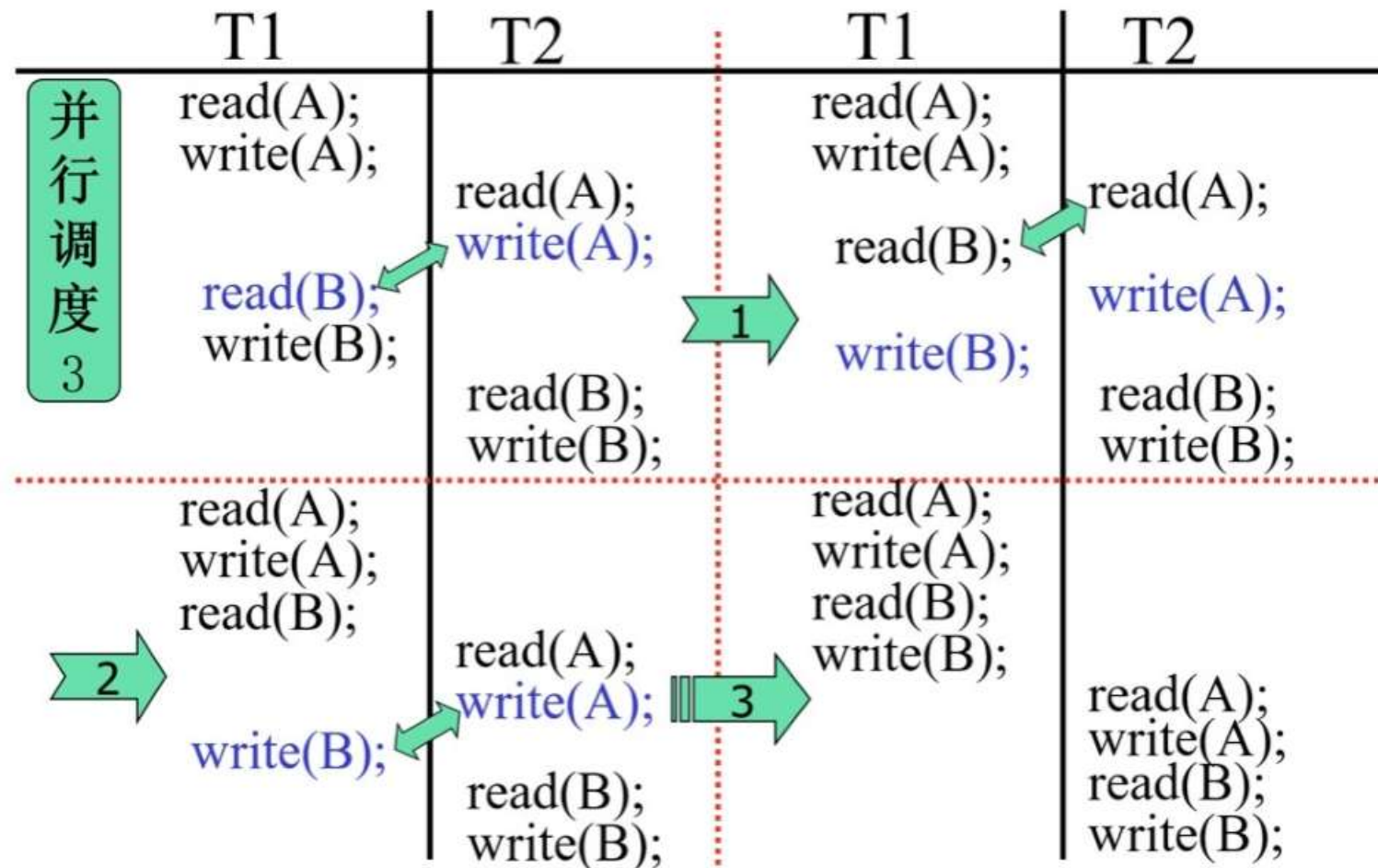
T_1	T_2
read(A)	
write(A)	
	read(A)
read(B)	
	write(A)
write(B)	
	read(B)
	write(B)

S5

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

S6

Conflict Serializability



Conflict Serializability

■ Note

- **Only swapping** of two operations/instructions I_i and I_j that belong to two **different** T_i and T_j respectively are permitted
- **Swapping should not** change **the orders** of instruction in **each** transaction
 - swapping of **read**(B) and **write**(B) in T_1 is **not** allowed
- **Swapping should not** change **orders** of **conflict instruction** executing **S**
 - swapping of **write**(A) in T_1 and **read**(A) in T_2 is **not** allowed

Conflict Serializability

- **Def:** A **concurrent** schedule S is **conflict serializable** if it is **conflict equivalent** to a serial schedule S'
 - e.g. concurrent $S5$ is equivalent to $S6$
- There are some schedules, which are not conflict serializable
 - e.g.

T_3	T_4
read (Q)	write (Q)
write (Q)	

Identification: Conflict Serializability

- **Identification:** whether a concurrent schedule S is conflict
- **Method 1**
 - starting from the concurrent S ,
maintaining executing orders of conflicting,
swapping executing orders of non-conflicting operations in S
 - observing whether a serial S' can be obtained
- **Method 2**
 - precedence-graph

Example 1

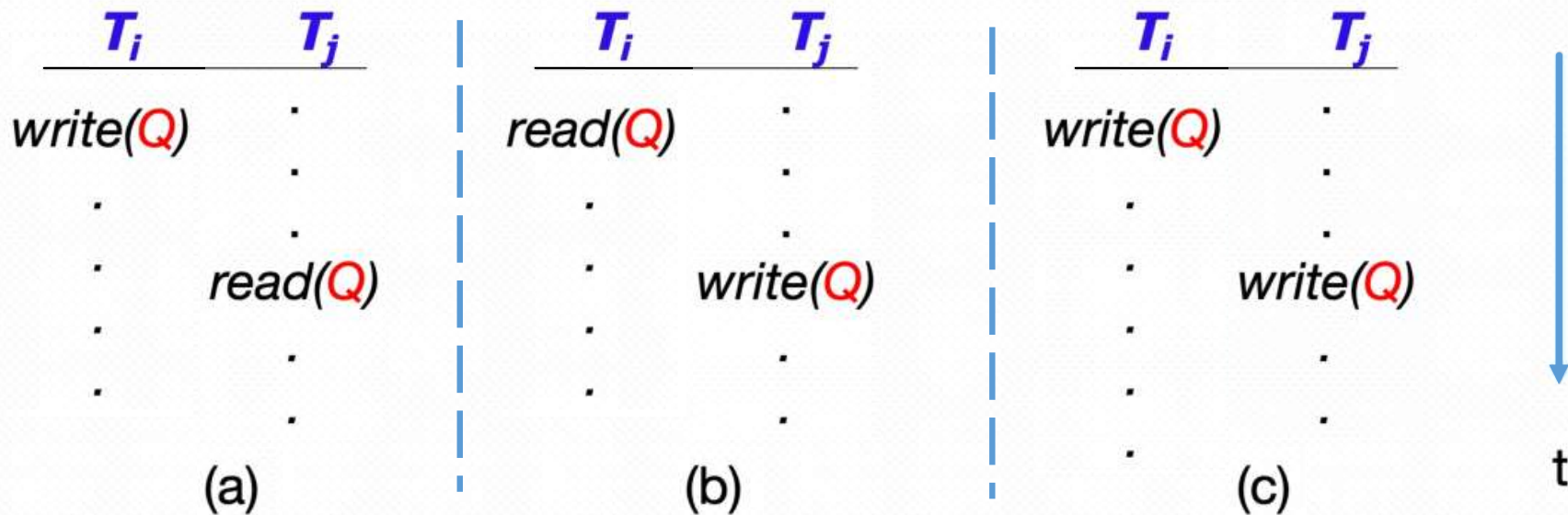
- Given the concurrent schedule S on transactions T1, T2, T3, and T4
 - ▣ Determine whether S is conflict-serializable
 - ▣ if S is conflict-serializable, give its equivalent serial schedule

Time	T1	T2	T3	T4
1			read(X)	
2		write(X)		
3			write(Y)	
4				read(Y)
5	read(Y)			
6				read(X)
7				write(Z)
8	write(X)			

- Answer: S is equivalent to <T3; T2; T4; T1>

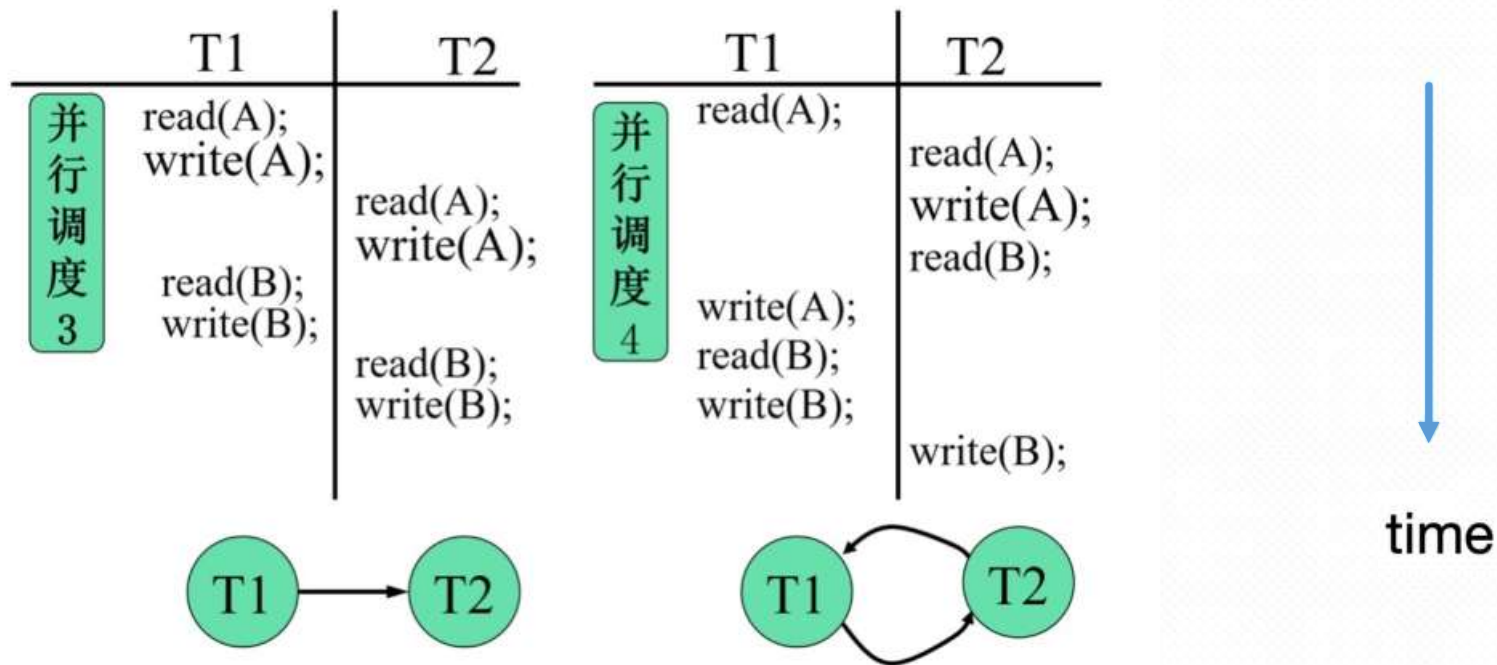
Precedence-graph

- **Def: precedence graph** (前趋图) $G(S)$ for the given transactions T_i, T_j ,
 - Direct graph $G(V, E)$, where the vertices are transactions
 - Arc from $T_i \rightarrow T_j$, for which one of three conditions holds
 - T_i executes $\text{write}(Q)$ before T_j executes $\text{read}(Q)$
 - T_i executes $\text{read}(Q)$ before T_j executes $\text{write}(Q)$
 - T_i executes $\text{write}(Q)$ before T_j executes $\text{write}(Q)$



Precedence-graph

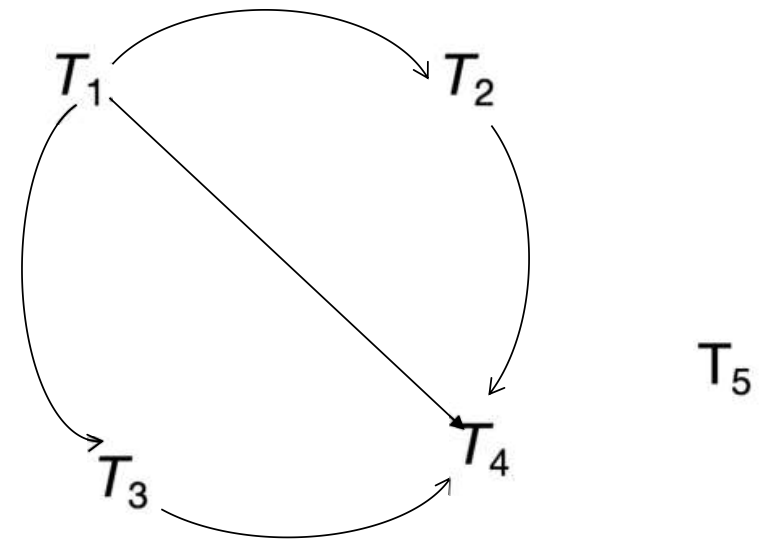
- **Def:** A schedule is **conflict serializable** if and only if its precedence graph is **acyclic**
- Example: Concurrent schedule S and its precedence graph





time

T ₁	T ₂	T ₃	T ₄	T ₅
read(Y) read(Z)	read(X)			
	read(Y) write(Y)			
read(U)		<u>write(Z)</u>		
			read(Y) write(Y) <u>read(Z)</u> write(Z)	
read(U) write(U)				read(V) read(W) read(W)



Construction of Conflict Serializable Schedule

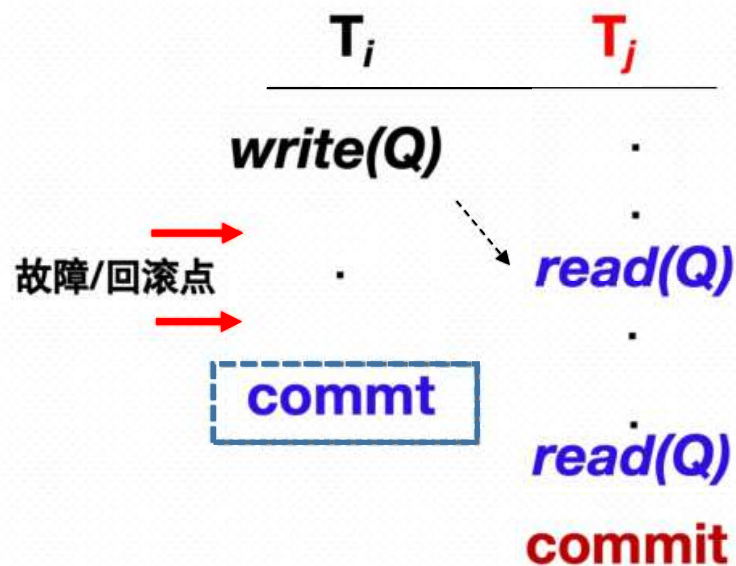
- Construct a **serializable S'** , conflict equivalent to a **serializable** schedule **S**
 - ▣ **starting** from a given schedule S ,
 - ▣ **maintaining** executing orders of conflicting operations in S ,
 - ▣ **swapping** executing orders of non-conflicting operations in S ,
 - ▣ **resulting** in a conflict serializable **S'**

17.7 Transaction Isolation and Atomicity

- **Two schedules** 【发生故障时，并发调度】
 - Recoverable scheduling
 - Cascadeless scheduling
- **Recoverability** when transaction failures
- **Requirements**
 - when transaction T_i fails, operations T_j that depends on T_i already executed should be rolled back/undone/aborted/canceled
 - T_j reads data having been written by T_i
 - ensure **atomicity**

Recoverability

- **Def: Recoverable schedule** is a schedule for each pair of T_i and T_j
 - if T_j reads data item Q previously **written** by T_i ,
commit operation of T_i **appears** before **commit** operation of T_j .



T_j 依赖于 T_i ;
当 T_i 中的 **write(Q)** 需要
aborted/rollback 时,
由于 T_i 还没有提交,
 T_j 也没有 commit,
因此 T_j 可以 rollback,
 T_j 中的 **read(Q)** 操作也
随 aborted/undo

Example 1: Non-recoverable schedule

- SC (s#, c#, grade)
check ($0 \leq \text{grade} \leq 100$)
- MaxGrade (c#, maxG)

s#	c#	grade
101	1	90
102	2	95

初始

s#	c#	grade
101	1	90
102	2	95
103	2	97

s#	c#	grade
101	1	95
102	2	100
103	2	97+5?

s#	c#	grade
101	1	90
102	2	95

T_i整体回滚结束后

引发update、insert回滚

check 约束:
0 ≤ grade ≤ 100

T_i:
begin transaction
insert into SC
value (103, 2, 97);

T_j:
begin transaction
insert into MaxGrade(c#, maxG)
select c#, max (grade)
from SC
group by c#
commit

不可恢复调度

update SC
set grade=grade+5;
commit

T_i结束后,
结果错误

c#	maxG

初始为空

c#	maxG
1	90
2	97

Rollback in SQL Server

- SQL Server利用语句和参数:

SET XACT_ABORT { ON | OFF }

指定当 Transact-SQL 语句产生运行时错误时，是否自动回滚当前事务

- ▣ 为 **ON** 时，如果 Transact-SQL 语句产生运行时错误，**整个事务**将终止并回滚
 - ▣ 为 **OFF** 时，只回滚产生错误的 Transact-SQL 语句，而**事务将继续进行处理**
- ▣ 默认情况SQL Server只Rollback出错语句（提高事务执行效率），而不是整个事务，即**SET XACT_ABORT OFF**

s#	c#	grade
101	1	90
102	2	95

初始

s#	c#	grade
101	1	90
102	2	95
103	2	97

s#	c#	grade
101	1	95
102	2	100
103	2	97+5?

s#	c#	grade
101	1	90/95?
102	2	95/100?
103	2	97

update回滚
insert未回滚

SET XACT_ABORT OFF

T_i:
begin transaction
insert into SC
value (103, 2, 97);

T_j

不可恢复调度

begin transaction
insert into MaxGrade(c#,maxG)
select c#,max(grade)
from SC group by c#
commit

update SC
set grade=grade+5;
commit

引发update回滚

check约束:
0<=grade<=100

**T_i结束后
结果正确**

c#	maxG
1	90
2	97

初始为空

s#	c#	grade
101	1	90
102	2	95

初始

s#	c#	grade
101	1	90
102	2	95
103	2	97

s#	c#	grade
101	1	95
102	2	100
103	2	97+5?

s#	c#	grade
101	1	90
102	2	95

SET XACT_ABORT ON

T_i:

begin transaction
insert into SC
value (103, 2, 97);
update SC
set grade=grade+5;

commit

T_j

begin transaction
insert into MaxGrade(c#,maxG)
select c#,max(grade)
from SC group by c#

commit

可恢复调度

T_i整体回滚结束后

引发update整体回滚

check约束:

0<=grade<=100

T_i结束后,
结果正确

c#	maxG

初始为空

c#	maxG
1	90
2	95

Example 2: Non-recoverable schedule

- S9 is **not recoverable** if T_9 **commits** immediately **after** the $read(A)$
 - if T_8 is aborted *after* T_9 has been committed, the value of the item **A** will be **rolled back** to its old value before T_8 executes.
 - **but** T_9 has read the new value of the data item **A** and has been **committed**

T_8	T_9
read (A)	
write (A)	
	read (A)
	commit
read (B)	
commit	

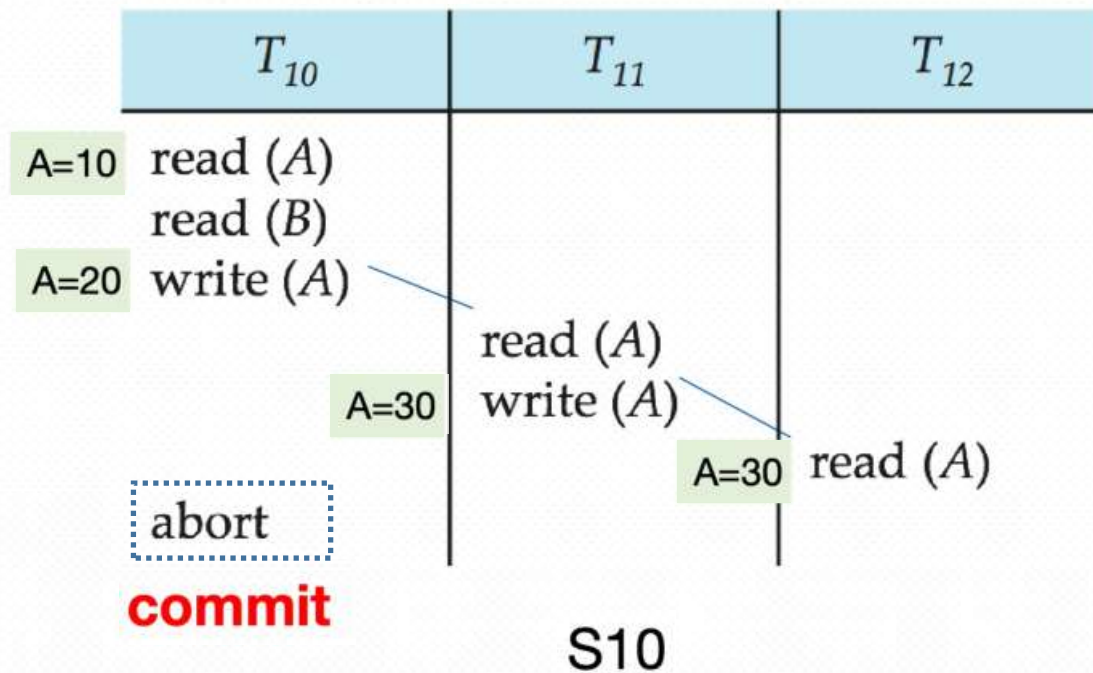
S9

Cascadeless Schedules

- **Def: Cascading rollback**

- a single transaction failure leads to series of transactions rollbacks

- **Before** T_{10} , T_{11} and T_{12} are committed, **after** T_{12} reads A , **when** T_{10} rollbacks due to failures, T_{11} should rollback, resulting in T_{12} rollback



s#	c#	grade
101	1	90
102	2	95

初始

s#	c#	grade
101	1	90
102	2	95
103	2	97

s#	c#	grade
101	1	95
102	2	100
103	2	97+5?

s#	c#	grade
101	1	90
102	1	95

T1整体回滚结束后

SET XACT_ABORT ON: 整体回滚

T₁₀:

begin transaction
insert into SC
value (103, 2, 97);

begin transaction
insert into MaxGrade(c#,maxG)
select c#,max(grade)
from SC group by c#

update SC
set grade=grade+5;
commit/rollback

c#	maxG
1	90
2	97

c#	maxG
1	90
2	95

rollback

T₁₂

级联回滚调度

begin transaction
select avg(maxG)
from MaxGrade

avgG
93.5

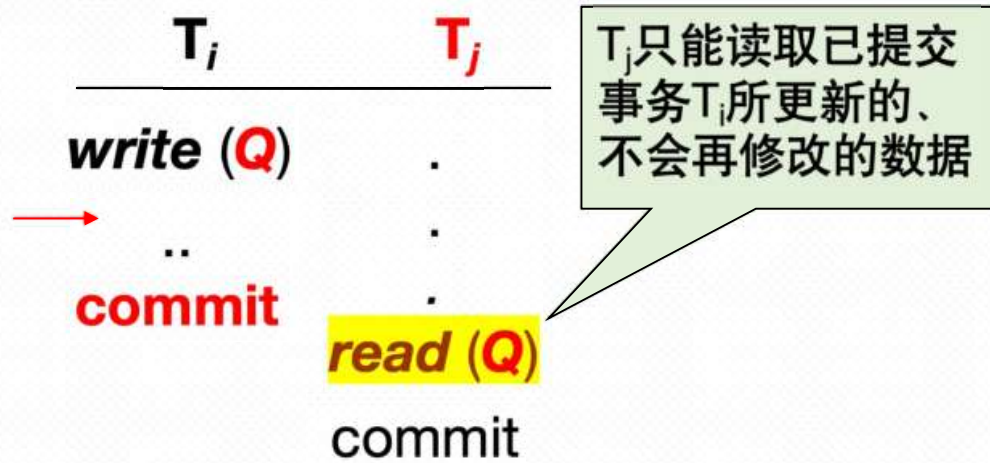
avgG
92.5

rollback

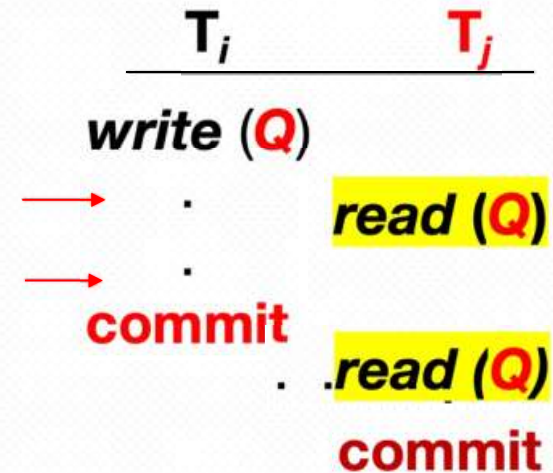
Cascadeless Schedules

- Cascading rollback is undesirable, because undoing of the previous works
- **Def: Cascadeless schedules** are the schedules that cascading rollbacks cannot occur;
 - for each pair of T_i and T_j , where T_j reads Q previously written by T_i , the **commit** operation of T_i appears **before** the **read** operation of T_j .
 - T_j 只读已提交事务 T_i 修改的数据, 不会再回滚发生变化的数据
 - **Cascadeless** schedules is expected for concurrent transactions

Cascadeless Schedules



Cascadeless schedule



Recoverable schedule

s#	c#	grade
101	1	90
102	2	95

初始

s#	c#	grade
101	1	90
102	2	95
103	2	97

s#	c#	grade
101	1	95
102	2	100
103	2	97+5?

s#	c#	grade
101	1	90
102	2	95

T₁₀整体回滚结束后

SET XACT_ABORT ON

T₁₀:

begin transaction
insert into SC
value (103, 2, 97);
update SC
set grade=grade+5;
commit

T₁₁

begin transaction
insert into MaxGrade(c#,maxG)
select c#,max(grade)
from SC group by c#
commit

c#	maxG
1	90
2	95

T₁₂

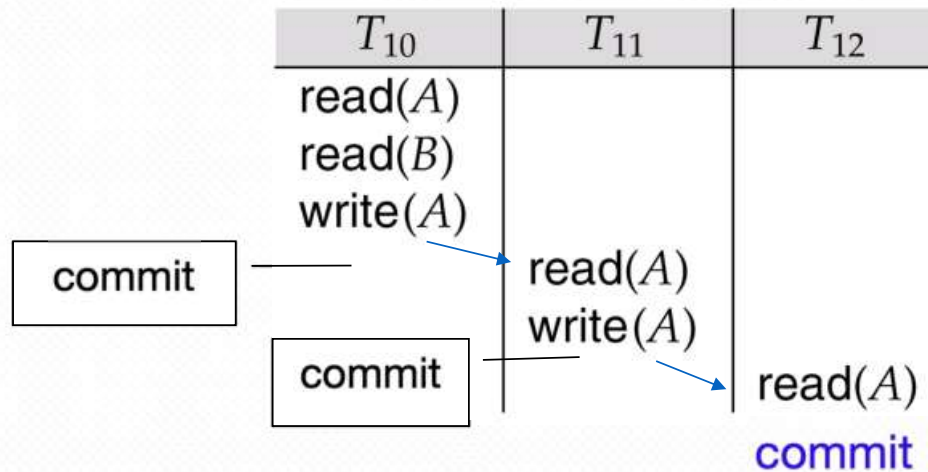
无级联回滚调度

avgG
92.5

begin transaction
select avg(maxG)
from MaxGrade
commit

Cascadeless schedule vs recoverable schedule

- Cascadeless schedule is also recoverable!
- But a recoverable may not be a cascadeless schedule
- **Example:** cascadeless and recoverable schedule



例题

- Consider transactions T_1 , T_2 , T_3 and T_4 under S , which concurrently access relational tables *Student* (*stuID*, *stuName*, *age*, *height*) and *Grade* (*stuID*, *cosID*, *grade*).
- Problem:
 - ▣ (1) Construct the precedence graph for S
 - ▣ (2) Is S a serializable schedule?
If not, give the reason.
 - ▣ (3) Is S a recoverable schedule, and why?
 - ▣ (4) Is S a cascadeless schedule, and why?

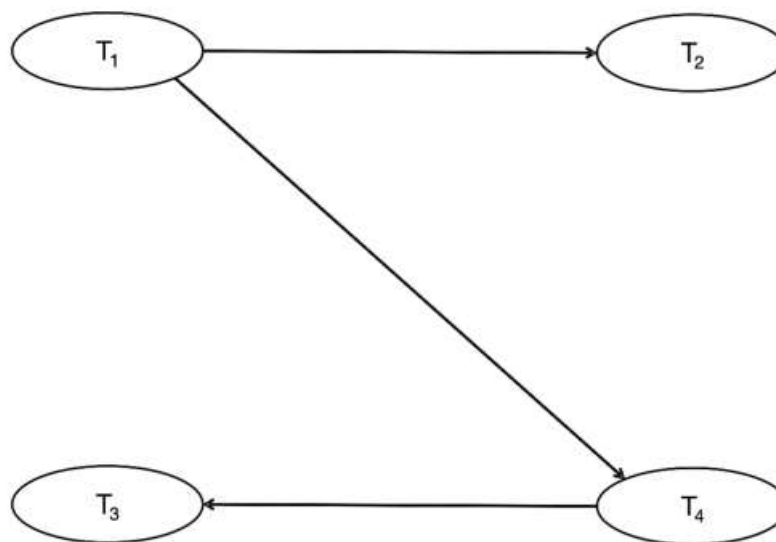
T ₁	T ₂	T ₃	T ₄
begin_transaction			
	begin_transaction		
update <i>Student</i> set <i>stuName</i> ='Li' where <i>stuID</i> =10			
		begin_transaction	
	update <i>Student</i> set <i>age</i> = <i>age</i> +1 where <i>stuID</i> =20		
		select <i>age</i> from <i>Student</i> where <i>stuID</i> =50	
			begin_transaction
			update <i>Student</i> set <i>height</i> = <i>height</i> +5 where <i>stuID</i> =30
update <i>Grade</i> set <i>grade</i> = <i>grade</i> +1 where <i>stuID</i> =40 AND <i>CosID</i> =10			
	select <i>stuName</i> from <i>Student</i> where <i>stuID</i> =10		
	commit		
commit			
			select <i>stuID</i> , <i>grade</i> from <i>Grade</i> where <i>stuID</i> =40 AND <i>CosID</i> =10
			commit
		Select <i>height</i> from <i>Student</i> where <i>stuID</i> =30	
		commit	

Example (cont.)

■ (1) 前驱图

- ▣ T_1 修改了student表中 $stuID=10$ 的元组, T_2 随后又读取该元组, 因此存在一条从 T_1 到 T_2 的冲突边
- ▣ 对其它各条冲突边可做类似分析

■ (2) 前驱图中无回路, 因此是冲突可串行



Example (cont.)

■ (3) 不是可恢复调度

- T_2 读取 student 表中 $stuID=10$ 的元组时，该元组内容已由 T_1 修改过，但 T_2 提交操作 `commit` 早于 T_1 提交 `commit`
- 一旦 T_1 在 T_2 的 `commit` 操作之后回滚其 `update student` 操作，将 `stuName` 回滚为旧值，则 T_2 的 `select` 操作无法随着回滚， T_2 读取的仍然是 `stuName` 修改后的值

Example (cont.)

- (4) 不是无级联回滚调度

- 因为不是可恢复调度，因此也不是无级联回滚调度
- 无级联回滚调度中，一个事务只能读取其它事务已经提交的数据。

但 T_2 读取student表中stuID=10的元组时，该元组被 T_1 修改，但未提交

Appendix 17-1 Transactions in SQL Server

- SQL Server系统中有三种事务执行模式：
显示、自动提交、隐性，用于启动事务执行

MySQL: *start transaction*

- Explicit transactions
 - ▣ delimited by ***begin-transaction*** and ***commit*** or ***rollback***
 - ▣ defined by user/application program, *explicitly controlled by user*
 - ▣ 工作在SQL Server系统的显示事务执行模式下 ▶
 - ▣ e.g. *account-transfer transaction*

Transactions in SQL Server

■ Auto-commit transactions

- SQL Server 默认的事务管理模式
- 在此模式下，每条单独的`Transact-SQL`语句均是一个事务
 - e.g.

CREATE, ALTER TABLE, SELECT, UPDATE, INSERT, DELETE,
DROP, FETCH, OPEN

- 每条语句完成时，可以成功提交(commit)，也可以失败滚回 (rollback)
- e.g. update SC

set Grade=Grade +10

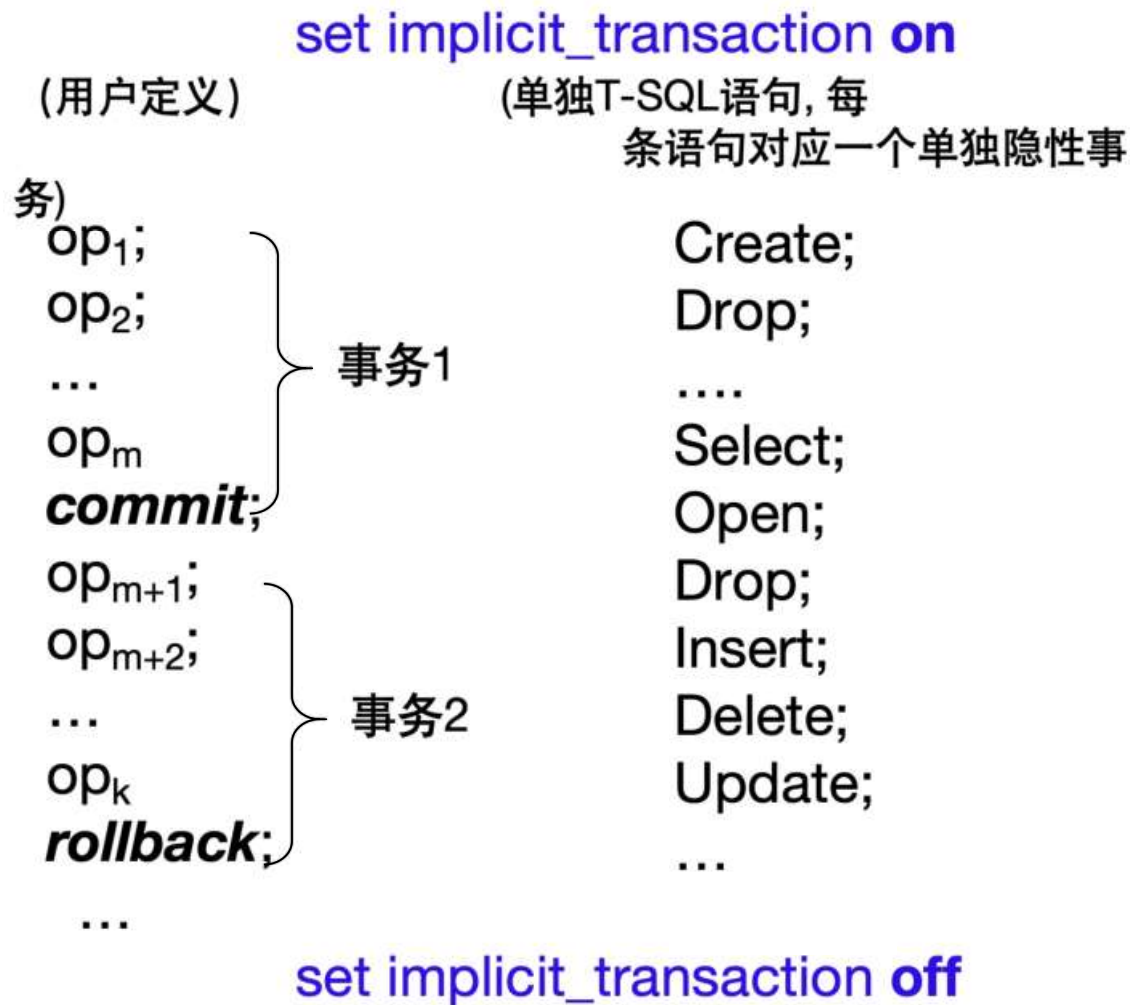
当某个学生Grade已经超过99，再执行本语句将导致事务回滚，SQL Server撤销整个更新语句的执行结果

Transactions in SQL Server

■ Implicit transactions /隐性事务

- SQL Server系统利用隐性事务模式设置命令设置系统的事务执行模式
set implicit_transaction on
- 在此模式下，每个事务无需（用begin-transition）显示地定义事务的开始，但仍以显示的**commit** or **rollback**结束
- 当一个事务结束后，系统自动启动下一个事务，无需begin-transition，形成连续的事务链

隐性事务执行模式



SQL Server 批处理中的事务

- 批处理
 - ▣ 包含1个或多个SQL语句的组，由应用程序从客户端一次性地发送到服务器端的DBMS引擎去执行
- 事务与批之间多对多关系
 - ▣ 1个事务可以包括多个批，1个批中也可以由多个事务组成
- SQL Server系统中，在大多数情况下，当批中的某些SQL语句执行发生错误时，DBMS将停止执行当前语句及其后面的语句，但发生错误之前已经执行的语句是有效的，不会被回滚(rollback)

SET XACT_ABORT OFF

Check 语义约束: grade between 0 and 100

begin tran

insert into SC values ('s1', 'c1', 90)

update SC

set GRADE=GRADE + 12

commit tran

select * from SC

□ -----

update 操作被回滚;

但Insert 操作未被回滚, ('s1', 'c1', 90)被成功插入;

不满足事务原子性要求! Why?

SET XACT_ABORT OFF

SQL Server 批处理中的事务

- SQL Server 中，如果需要事务中的全部操作/语句作为1个整体，同时提交或回滚，在提交“批”之前，采用下述语句进行设置：

```
set XACT_ABORT ON
```

```
begin tran
```

```
    insert into SC values ('s1', 'c1', 90)
```

```
    update SC
```

```
    set GRADE=GRADE + 12
```

```
commit tran
```

```
select * from SC
```

- update 操作、insert操作均被回滚，('s1', 'c1', 90)无法插入

Appendix 17-2 Savepoint

- SAVEPOINT 语句定义事务执行过程中的中间点/状态
 - ▣ 使用保存点将各组相关语句分开可以在事务内标识重要的状态
- 使用 ROLLBACK TO SAVEPOINT 语句可以撤消该点后的所有更改，即将事务的执行退回到以前的状态

```
BEGIN TRANSACTION
USE student-DB
INSERT INTO student
VALUES ( " 03402", "王菲", "CS", "2000/05/15")
SAVE TRAN My-savepoint          /* defining save-point */
DELETE FROM student
WHERE name= "王菲" or "章立"
ROLLBACK TRAN My-savepoint
COMMIT TRAN
GO
```

Fig.17.0.14 An example of savepoint in T-SQL

■ Note

- rollback将操作滚回到保存点 **My-savepoint**: delete 操作被**rolled back**, 而insert操作则不被**rolled back**
- DB恢复为delete操作执行前的状态, 新插入的元组 ("03402", "王菲", "CS", "2000/05/15")并未被删除, 仍然保存在数据库中

st-id	st-name	depart- ment	birth- date
03405	章立	CS	2000/01/25
...	...	...	...
03409	李龙	CS	2000/12/20
...	...	...	...
03411	赵新	CS	2000/06/18

(a) DB before the transaction starts

st-id	st-name	depart- ment	birth- date
03402	王菲	CS	2000/05/15
03405	章立	CS	2000/01/25
...	...	...	...
03409	李龙	CS	2000/12/20
...	...	...	...
03411	赵新	CS	2000/06/18

(b) DB after ***insert*** is issued

Fig.17.0.15-I DB instances at different timepoints

st-id	st-name	department	birth-date
...	...	...	...
03409	李龙	CS	2000/12/20
...	...	...	...
03411	赵新	CS	2000/06/18

(c) DB after ***delete*** is issued

st-id	st-name	department	birth-date
03402	王菲	CS	2000/05/15
03405	章立	CS	2000/01/25
...	...	...	...
03409	李龙	CS	2000/12/20
...	...	...	...
03411	赵新	CS	2000/06/18

(d) DB after ***rollback*** is issued

Fig.17.0.15-II DB instances at different timepoints

Chapter 17 Transactions: Outline

